CNCF - Headlamp (2025 Term 1) Build Plugin Installation Container

1 Table of Contents

1 Table of Contents 1
2 About 2
3 Problem Statement 2
3.1 Background 2
3.2 Objective 2
3.3 Interest 2
4 Implementation and Deliverables 3
4.1 Current Headlamp plugins CLI 3
4.2 Plugin Configuration Format 3
4.3 CLI Enhancements 3
4.4 Config validation 3
4.5 Batch installations 4
4.6 Monitoring and Logging 4
4.7 Containerization 4
4.8 Support for Helm Charts 5
4.8.1 Basic Architecture Diagram 5
4.8.2 Volumes 5
4.8.3 Config Maps 6
4.8.4 Init Container 6
4.9 Add CI/CD Support 7
4.10 Backup and Restore Strategies 7
4.11 Testing 7
4.12 Documentation 8
5 Project Timeline and Milestones 9
6 Core Implementation Prototype (Must Read Section) 10
7 Solved Issues (Must Read Section) 10
8 More About Me 11
8.1 Experience 11
8.2 Previous Projects 11
8.3 Open Source 11
8.4 Background and Education 11
8.5 LFX Experience 11
9 References 12

2 About

Name	Faakhir Ul Hassan
Email	zahidfaakhir@gmail.com fzahid.bscs22seecs@seecs.edu.pk
Slack	Slack – Faakhir30 (CNCF, Kubernetes)
Profiles	LinkedIn - https://www.linkedin.com/in/faakhir GitHub - https://github.com/faakhir30
Project Mentors	René Dudfield, Kautilya Tripathi, S Santhosh Nagaraj

3 Problem Statement

3.1 Background

Headlamp currently supports plugin installation through the headlamp-plugin CLI tool, which can install plugins to a specified directory. However, for in-cluster mode deployments, we need a more Kubernetes-native way to manage plugins that can:

- Install plugins during container initialization
- Support configuration management
- Handle plugin updates and removals
- Work seamlessly with Kubernetes deployments

3.2 Objective

Create a containerized version of the headlamp-plugin CLI tool that can be used as an init container in Kubernetes deployments. This container will:

- Accept a configuration file (via ConfigMap) specifying which plugins to install
- Install the specified plugins into a shared volume before Headlamp starts
- Be automatically built and published as part of Headlamp's release process
- Provide a seamless way to manage plugins in Kubernetes environments

The container will be used in Headlamp's Helm chart as an init container, allowing operators to declaratively manage their plugins through Kubernetes-native configuration.

3.3 Interest

My interest in this project stems from my passion for Kubernetes and web technologies. Having contributed to Headlamp's frontend and plugin system, I've gained valuable insights into its architecture and the challenges faced by users when managing plugins.

The opportunity to improve the plugin management system aligns perfectly with my technical skills and desire to make Kubernetes tooling more accessible and user-friendly.

4 Implementation and Deliverables

4.1 Current Headlamp plugins CLI

Current Headlamp plugins CLI does provide quite a configurable setup for installing single plugin with ability to specify headlamp version, output directory etc. However, it does not support batch plugins installations and management.

4.2 Plugin Configuration Format

Implementation of a YAML-based configuration format that supports:

- Plugin source URLs and versions
- Plugin-specific configurations
- Post-installation hooks (if any)

Here's a sample config:

```
- name: app-catalog
source: "https://artifacthub.io/packages/headlamp/test-123/appcatalog_headlamp_plugin"
version: "0.5.0"
config: {}
postInstall:
- "echo “hello headlamp”"
```

This format will also be later followed in values.yaml for management of plugins in-cluster.

4.3 CLI Enhancements

CLI commands would be improved to support install from our specified config file.

Here's sample plugin cli install command's expected state:

```
./headlamp-plugin.js install --help
headlamp-plugin.js install
Install plugins
Options:
  --version      Show version number[boolean]

   --help        Show help[boolean]

-c, --config      Path to plugin configuration file[string]

-d, --plugins-dir  Directory to install plugins[string] [default: "/headlamp/plugins"]

-s, --source      Plugin source URL (for single plugin installation) [string]
```

4.4 Config validation

I plan to use the popular **ajv** module config validation, with or without the **yaml-validator** module.

This module will also be responsilble for security perspective of the source urls, it should validate that source urls do validate with proxy-urls (https://artifacthub.io/*, etc)

```
class ConfigValidator {
 constructor() {
  this.ajv = new Ajv({ allErrors: true });
  this.validate = this.ajv.compile(pluginConfigSchema);
 }
```

4.5 Batch installations

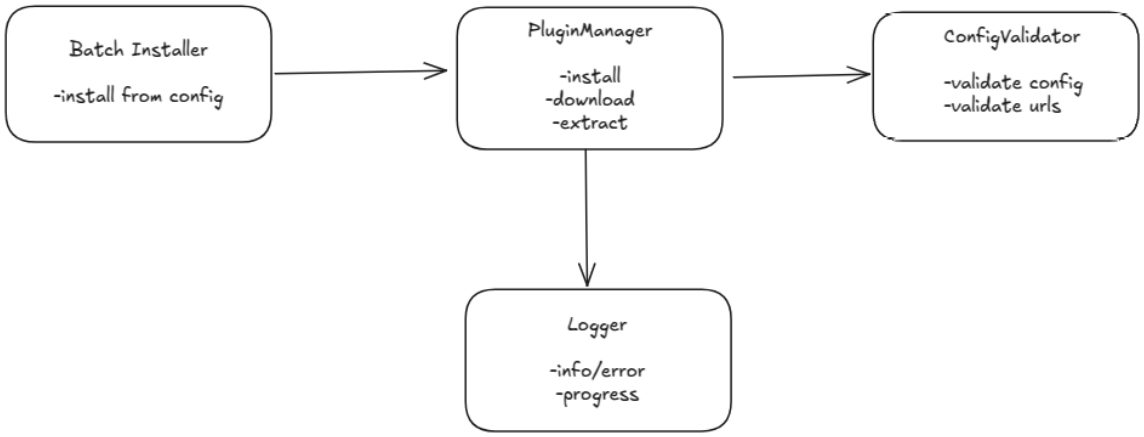
A batch-installer.js module would be implemented to manage installations, updates, removal of plugins to use if config (-c) cli param is provided:

```
class BatchInstaller {
 constructor(pluginsDir = process.env.PLUGINS_DIR) {
  this.pluginsDir = pluginsDir;
  this.configValidator = new ConfigValidator();
  this.logger = logger.child({ component: 'BatchInstaller' });
 }
```

This module would support following notable functionalities:

Install From Config: Install plugins from a configuration file

Apply Plugin Config: Apply configuration to an installed plugin



4.6 Monitoring and Logging

A logger.js module might be required to implement logs, didn't saw any logger module in plugins/headlamp-plugin. This would be required to provide proper logs for supporting easy debugging in cluster deployment.

A decent option is to use **winston** library to enhance monitoring of the installation process. It would provide capability to have timestamps, clean stack traces, Json formats, writing logs to files (e.g., error.log, exceptions.log, ..)

4.7 Containerization

For being able to use this headlamp-plugin CLI tool in-cluster, containerization is required. Containerization would mainly include:

- Set up Docker build process
- Create installation scripts

- Include necessary dependencies

However, optimizing this build image is also important, so far, I've tried and tested with node:20-alpine3.19 as base image for both build and production stage. But this image is resulting in a much larger size of around 900MB even with a multi-stage Dockerfile.

A key optimization would be choosing the base image more carefully, I believe a CLI shouldn't require that heavy base image.

4.8 Support for Helm Charts

4.8.1 Basic Architecture Diagram

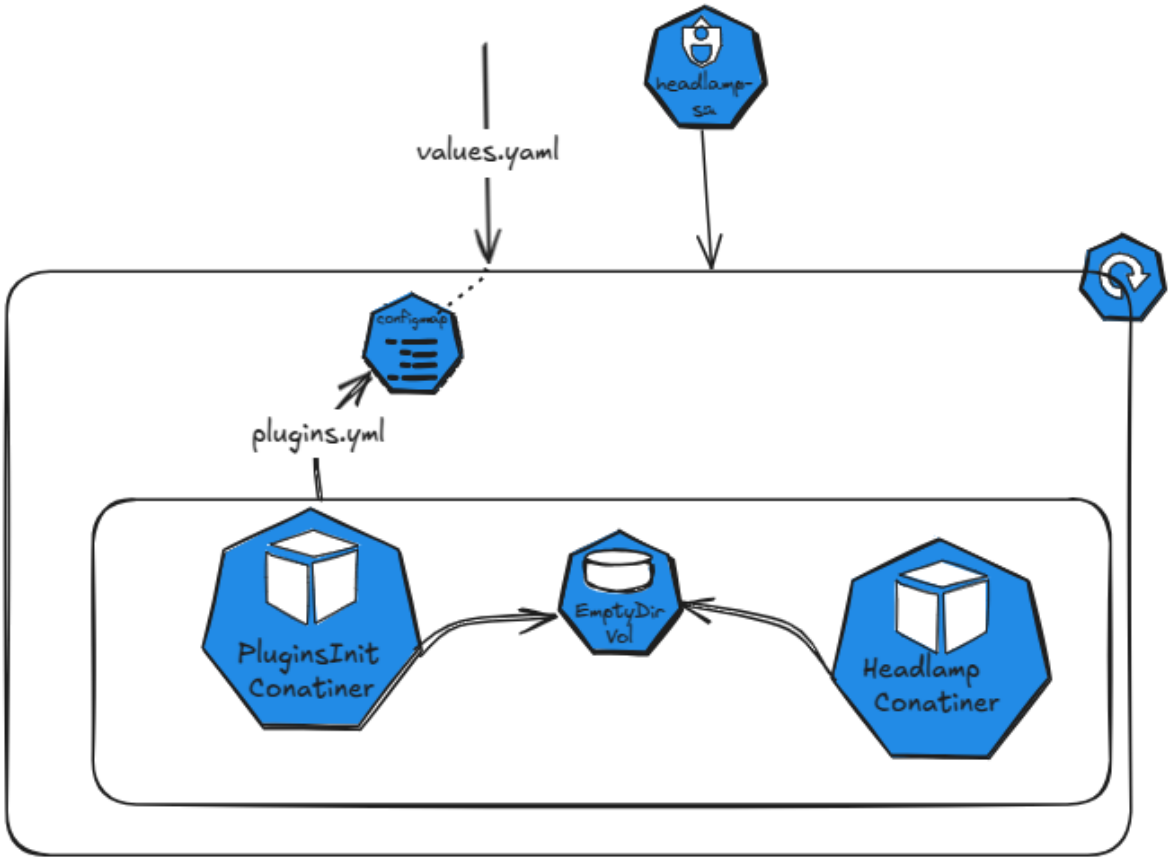


Fig 1. In-Cluster Deployment with emptyDir Volume.

4.8.2 Volumes

An easy way to do this is to use Pod volumes with emptyDir config.

```
# -- Headlamp containers volume mounts
volumeMounts:
- name: plugins
mountPath: /headlamp/plugins
- name: plugin-config
mountPath: /config
readOnly: true

# -- Headlamp pod's volumes
```

```
volumes:
- name: plugins
emptyDir: {}
- name: plugin-config
configMap:
name: headlamp-plugins-config
```

This does get the job done in single cluster deployments. It is Simple and lightweight, no extra storage provisioning needed.

However, in multi-cluster deployments, this emptyDir wont persist across clusters. A PV ensures that the plugin data is retained across restarts and even rescheduling.

```
volumes:
- name: plugins
persistentVolumeClaim:
claimName: headlamp-plugins-pvc
- name: plugin-config
configMap:
name: headlamp-plugins-config
```

4.8.3 Config Maps

A ConfigMap will be used for centralized, declarative, and automated plugin installation in Kubernetes. It simplifies plugin management, especially in production environments.

```
{{- if .Values.pluginInstaller.config.enabled }}
apiVersion: v1
kind: ConfigMap
metadata:
name: {{ .Values.pluginInstaller.config.name }}
labels:
{{- include "headlamp.labels" . | nindent 4 }}
data:
plugins.yaml: |
{{- .Values.pluginInstaller.config.content | toYaml | nindent 4 }}
{{- end }}
```

Now, the init container's entry point would look something like following:

```
CMD ["sh", "-c", "node bin/headlamp-plugin.js install --config /config/plugins.yaml]
```

4.8.4 Init Container

This will download and setup plugins in volume, before initializing the headlamp container. So that headlamp container can load plugins from there.

```
{{- if .Values.pluginInstaller.config.enabled }}
- name: plugin-installer
image: "{{ .Values.pluginInstaller.image.repository }}:{{ .Values.pluginInstaller.image.tag }}"
imagePullPolicy: {{ .Values.pluginInstaller.image.pullPolicy }}
env:
- name: PLUGINS_DIR
value: "/headlamp/plugins"
- name: CONFIG_DIR
```

```
value: "/config"
- name: LOG_LEVEL
value: "info"
volumeMounts:
- name: plugins
mountPath: /headlamp/plugins
- name: plugin-config
mountPath: /config
readOnly: true
securityContext:
{{- toYaml .Values.securityContext | nindent 12 }}
{{- end }}
```

4.9 Add Cl/CD Support

A general workflow needed for CI/CD:

- **image builds:**
  - Would be creating a github actions workflow to automate build to ghcr.io or any other.
  - Build latest images on main branch updates.
  - Build specific versions when a release tag (e.g., v1.0.0) is pushed.
  - Use Git Tags (vX.Y.Z) to Trigger Versioned Builds.
- **Container Scanning:**
  - Will be using Trivy or Snyk to Scan for CVEs.
  - Fail the CI/CD pipeline if high-risk vulnerabilities are found.
  - Run periodic security scans on existing images.
- **Release Automation:**
  - Automatically create GitHub Releases from tags.
  - Attach changelogs and Docker image references to releases.
  - Integrate with Helm Charts to deploy new images automatically.

4.10 Backup and Restore Strategies

PersistentVolume and add an option in configmap and values.yaml to allow create a kubernetes CronJobs to automate backups at regular intervals.

Alternates:

- Volume Snapshotting: Allows automated backups of the plugin storage. Use Kubernetes VolumeSnapshots to periodically capture the state of the plugins PVC.
- ConfigMap Backup: ConfigMaps are not automatically backed up, so we must store versions externally.Export the ConfigMap periodically:

```
kubectl get configmap headlamp-plugin-config -o yaml > backup-config.yaml
```

4.11 Testing

- we will perform multiple layers of testing, including unit testing, integration testing and e2e testing.

- Validate individual components such as configuration parsing, plugin installation logic, and error handling.
- Basic Test Cases:
  - Verify that a valid YAML config is parsed correctly.
  - Validate incorrect/malformed YAML handling.
  - Test installation of a single plugin.
  - Test installation of multiple plugins.
  - Ensure proper error messages when a plugin URL is unreachable.
  - Validate handling of missing/incorrect versions.

E2E Testing:

Although, e2e testing is not mentioned in upstream issue itself, but as a **stretch goal** I would like to consider e2e testing for headlamp plugin CLI in-cluster deployment with init-container.

Test the complete flow from **configuration → plugin installation → Headlamp pod bootup → plugin volume**. One option is to do this using **kuttl.dev** in GA workflows.

Common Test Cases:

- Deploy Headlamp with plugins specified in the ConfigMap.
- Verify that installed plugins appear in the UI and function correctly.
- Ensure the Headlamp UI doesn't break if a plugin installation fails.
- Test plugin updates: Modify the ConfigMap and restart the pod.
- Verify plugin removal by removing it from the ConfigMap and restarting.

4.12 Documentation

If it is not documented, it is not done.

A complete flow for deployment with plugin management via helm charts shall be documented.

5 Project Timeline and Milestones

TABLE 1 - TIMELINE LFX 2025

Feb 27 - March 2 *Community Bonding Period*	Solving common headlamp issues with fellow community members and getting to know them. Analysis of plugins deployments. Review existing plugins documentation. Review plugins-management code. This will help me in later implementing the project.
March 3 – March 10 *Coding Period*	Construct a Plugin Configuration Format. Write a configuration parsing functionality. Document the template.
March 11 - March 17 *Coding Period*	Write Batch Installer Module to install all the parsed config. Implement CLI enhancements to accept config file.
March 18 - March 23 *Coding Period*	Containerize the CLI optimally. Test installations from docker to a mounted volume.
March 29 – April 14 *Coding Period*	Add support in Helm Charts using volumes, configMaps And Init containers. Implement logging and monitoring for plugin installation status.
April 12 – April 15 *Evaluation*	**First Evaluation.** Preparation for evaluation and parallel Coding.
April 15 - April 17 *Coding Period*	Support PV (Persistent Volumes) for production.
April 18 - April 24 *Coding Period*	Work on backup and restore strategies for plugins config.
April 25 – May 6 *Evaluation*	Add CI/CD Support with GitHub actions. Add image versioning, container scanning. Configure release automation.
May 6 - May 12 *Coding Period*	Document deployment in-cluster with plugins config values for easy management.
May 12 - May 19 *Coding Period*	Final bug fixes and wrap up. Completion of deliverable features.
May 20 – May 27 *Code Submission and Evaluation*	**Final Evaluation**. Preparation for evaluation. Submission of project code and files.

Note: I have already skipped March 24–28 in the above timeline for my university exams, since I won't be available full-time during that period. However, I will make an effort to compensate for any impact over the weekends.

I plan to work 40 hours per week for LFX activities. I will not have any other full-time job during the period at LFX so I will be able to fully concentrate on the project. I will constantly keep in touch with the mentor with every milestone so the work will be thoroughly monitored.

6 Core Implementation Prototype (Must Read Section)

As a proof of concept for myself, I also did implement prototype for this proposals core implementation. However, it does not follow best coding practices, security considerations. Here's a draft PR: **https://github.com/Faakhir30/headlamp/pull/2**. This PR also contains a video to share the progress state I implemented as proof of concept.

7 Solved Issues (Must Read Section)

I am familiarizing myself with the Headlamps codebase by working on issues and following the developer guides available. I have worked on multiple issues on GitHub to get a good grasp of headlamp.

- **Plugins:** added Resource CircularChart example plugin
  - Learnt how to create plugins and integrate them in frontend, created new processors, plugin registering functionality, redux slices for plugins.
  - **Pull Request:** https://github.com/headlamp-k8s/headlamp/pull/2832

- **Apps, Frontend:** refactor and test Rename Cluster in settings
  - **Pull Request:** https://github.com/headlamp-k8s/headlamp/pull/2871

- **Frontend:** Add Storybook Stories for Notifications Component
  - **Pull Request:** https://github.com/headlamp-k8s/headlamp/pull/2829

- **Backend:** Refactor refreshAndCacheNewToken in headlamp.go
  - **Pull Request:** https://github.com/headlamp-k8s/headlamp/pull/2816

- **Docs:** Fix contributing doc test and formatting instruction
  - **Pull Request:** https://github.com/headlamp-k8s/headlamp/pull/2834

- **Frontend:** add Chooser componnent stories
  - **Pull Request:** https://github.com/headlamp-k8s/headlamp/pull/2824

- **Frontend:** global searchbar: Fix multiple clusterSettings items
  - **Pull Request:** https://github.com/headlamp-k8s/headlamp/pull/2907

- **Frontend:** Add clusterChooser componnent stories
  - **Pull Request:** https://github.com/headlamp-k8s/headlamp/pull/2823

- **Frontend:** Add stories for chart componenets
  - **Pull Request:** https://github.com/headlamp-k8s/headlamp/pull/2821

8 More About Me

8.1 Experience

I have been working as a Full Stack Engineer (Backend Focused) for the past 3 years. I have expertise in building scalable data pipelines, cloud infrastructure, and automation. Experienced in AWS, GCP, Docker, and CI/CD, with a strong foundation in TypeScript and python. I also have foundational knowledge about Golang and am willing to learn Golang for production level systems also.

8.2 Previous Projects

I have worked on projects mainly in JavaScript/TypeScript, Python and Golang. All of them are available on my GitHub profile, for short overview of my projects, you can view my site https://faakhir.tech/ . All those project's frontends are written in React or NextJS with typescript, and backend is written in python/golang. Currently working on Upautomate.ai https://upautomate.ai (Associated with BleedAI company).

8.3 Open Source

I have a keen interest in open source since it is such a wonderful community which contributes to the world by developing free software and letting people help each other with their projects.

I am relatively new to the community. However, I am now actively a part of it and have contributed to multiple repositories within CNCF, Plone and other organizations in the past months.

8.4 Background and Education

I am a 6th Semester student at National University of Science and Technology (NUST), Islamabad and am pursuing bachelor's in software engineering for the past 3 years.

I am a resident and native of Pakistan, currently living in the Islamabad, capital of Pakistan. I will be spending my all-time of this term in the same city at my home. I will be sure to keep in intense contact with my mentor to keep the process transparent and report regularly with the progress.

8.5 LFX Experience

This is the first time I am taking part in LFX.

I also planned initially to apply also for the KEDA plugins, but due to time constraint i could not write a detailed proposal for it. So, I am only applying for this Plugins CLI and its in-cluster management enhancement project.

I decided to apply for the LFX because of my passion to learn and support the open-source community. I want to contribute my share to the open-source world so that other people and software might benefit from it. Furthermore, I decided to apply to this project because I have a personal interest in backend systems, automation and devops related work.

I am a hard-working person, always looking forward to and challenging myself, with a genuine passion for the open-source community and improving my skills.

9 References

- https://excalidraw.com/
- https://kuttl.dev/
- https://trivy.dev/latest/
- https://www.npmjs.com/package/ajv
- https://www.npmjs.com/package/yaml-validator
- https://www.npmjs.com/package/winston